

LOOPGYM: TARGET-HIDDEN LOOP VERIFICATION WITH A SHARED ROLLOUT-HOUDINI FRAMEWORK

Anonymous authors

Paper under double-blind review

ABSTRACT

Verifying a loop requires more than producing a predicate that is inductive: the predicate must retain enough information to establish the condition required after the loop. Most invariant-generation systems expose this postcondition or assertion to the generator, allowing candidates to be tailored to, or copied from, the target. We study *target-hidden loop verification*. The generator sees the precondition, loop guard, and body but not the target Q ; it emits ACSL loop invariants I , which are accepted only when Frama-C/WP proves both their inductiveness and the original exit obligation $I \wedge \neg B \Rightarrow Q$. LOOPGYM organizes learning and inference around one rollout-Houdini framework. Both form groups of invariant candidates, merge or repair their clauses, and apply the same Frama-C/WP Houdini operator. The training objective attaches dense credit to this core: a rollout is scored by standalone negative rejection and extra credit for hard trace units rejected by few peers in its rollout group. A conservative semantic duplicate cost and response overflow cost discourage wasted clauses without penalizing unique proof-supporting clauses. A repair is scored by its gain over the merged pool. Every policy response obeys a configurable clause cap, with explicit overflow cost. Inference executes the same rollout, pool-growth, feedback, and pruning stages, then restores Q for the final proof. This design aligns the model actions and prover-backed survivor semantics optimized during training with those executed at deployment.

1 INTRODUCTION

Loop invariants are the linchpin of deductive program verification. They summarize an unbounded computation in a finite assertion, connecting the operational behavior of a loop to the mathematical facts needed by a proof. Generating them therefore demands both *program understanding*—tracking control flow, assignments, and interacting recurrences—and *mathematical reasoning*—discovering relations and bounds that hold across arbitrarily many iterations. Ever since the inductive-assertion method of Floyd (Floyd, 1967) and the axiomatic semantics of Hoare (Hoare, 1969), verifying a loop $\text{while } (B) S$ against a contract $\{P\} \cdot \{Q\}$ has reduced to exhibiting an *inductive invariant* I satisfying initiation ($P \Rightarrow I$), consecution ($\{I \wedge B\} S \{I\}$), and safety ($I \wedge \neg B \Rightarrow Q$). The goal is not merely to find a predicate that satisfies the first two obligations. An invariant such as *true* may be inductive but carries no information to the loop exit. Useful loop reasoning must preserve enough information to establish the required post-loop condition Q . This final obligation is the distinction between generating arbitrary invariants and completing loop verification.

When the desired post-loop condition Q is available, invariant synthesis can exploit it directly as a search objective. Template- and constraint-based procedures, counterexample-guided inductive synthesis (CEGIS), ICE-style learners, and postcondition-guided inference can use Q , failed proof obligations, or counterexamples to steer the search toward an invariant sufficient for Q (Gulwani et al., 2008; Gupta & Rybalchenko, 2009; Dillig et al., 2013; Alur et al., 2013; Garg et al., 2014; 2016; Nguyen et al., 2017; Furia & Meyer, 2010). Recent systems further combine this verifier-guided loop with neural or large language model (LLM) proposals, filtering, ranking, and repair (Si et al., 2020; Kamath et al., 2023; Chakraborty et al., 2023; Wu et al., 2024; Cao et al., 2025; Liu et al., 2024b). However, these formulations presuppose that Q has already been written, whereas obtaining a useful post-loop condition is often itself a nontrivial specification task. For a *while* loop, an invariant naturally induces such an exit condition when combined with the negated guard. We therefore reverse

the usual direction: without exposing the ground-truth target Q , we generate an invariant I from the loop and define its induced post-loop condition as

$$Q' \triangleq I \wedge \neg B.$$

We then evaluate whether the generated condition is strong enough by checking $Q' \Rightarrow Q$, using Q only as held-out ground truth rather than as input to synthesis.

Checking a proposed invariant and discovering one are markedly different problems. Once a candidate is fixed, an automated theorem prover (ATP) can often discharge its verification conditions. Generation must first invent the right abstraction from an open-ended space of predicates: it must be weak enough to hold initially and survive every iteration, yet strong enough to imply a downstream property. This tension has made automatic loop-invariant generation a longstanding and difficult problem.

Why numerical loops? We focus on numerical loops for two reasons. First, candidate invariants in this domain are predominantly arithmetic equalities and inequalities, often polynomial ones. Their correctness is comparatively amenable to automatic checking, so ATP feedback can serve as a scalable and objective signal without requiring the interactive proofs commonly needed for richer program states. Second, numerical loops largely factor out the representational machinery of arrays, heaps, and recursive data structures, whose verification often depends on inventing auxiliary functions or inductive predicates. They therefore provide a controlled setting in which success depends more directly on algebraic discovery and reasoning about loop dynamics, rather than on reproducing a particular data-structure abstraction.

This restriction does not make the problem easy. The Collatz iteration, for example, is expressible as a one-variable loop with two elementary arithmetic updates, yet whether it reaches 1 from every positive integer remains open (Lagarias, 1985). Consequently, no known polynomial invariant, together with an automatically checkable descent argument, suffices to certify this global claim; such a certificate would resolve the conjecture. Even within this deliberately simple domain, compact program syntax can thus hide mathematical behavior beyond presently known invariants.

From algorithmic to learned generation. Classical algorithmic methods have made their strongest exact progress on affine transition systems. Karr’s seminal procedure computes affine equalities, while abstract interpretation and constraint-based methods extend this foundation to linear equalities and inequalities (Karr, 1976; Cousot & Cousot, 1977; Cousot & Halbwachs, 1978; Colón et al., 2003). The canonical setting is a loop whose numerical state y evolves as

$$y' = Ay + b,$$

possibly with linear guards. Within this setting, invariant inference can exploit linear algebra, convex geometry, or decidable constraint solving. Methods based on polynomial templates and Gröbner bases extend the frontier to selected nonlinear algebraic invariants, but impose syntactic or algebraic restrictions and face an explicit trade-off between completeness and tractability (Sankaranarayanan et al., 2004). Thus, the most robust algorithmic guarantees remain concentrated around affine updates, rather than general nonlinear loop dynamics.

Data-driven and learned methods broaden the search by using traces, counterexamples, or neural generators (Garg et al., 2016; Si et al., 2020; Kamath et al., 2023; Cao et al., 2025). Many such systems expose Q to the generator and use its proof as feedback. This is effective for a fixed verification instance, but it creates a shortcut: a model can tailor candidates to the visible assertion without learning the relations and bounds that explain the loop. Postcondition-derived invariant mutation makes this dependence explicit (Furia & Meyer, 2010).

Motivating example. Consider `NLA_lipus/10.c`, abridged below:

```
1 int x = 1, y = 1, c = 1;
2 while (c < k) {
3   c = c + 1;
4   x = x*z + 1;
5   y = y*z;
6 }
7 /*@ assert 1+x*z-x-z*y == 0; */
```

The assertion itself is established by the initial state and preserved by the loop body. When it is visible, invariant generation collapses to copying the target: `loop invariant 1+x*z-x-z*y == 0;`. The resulting clause is inductive and proves the assertion without revealing why this relation follows from the two coupled recurrences. When the assertion is hidden, the same clause must instead be recovered from the assignments to `x` and `y`. This small example captures the distinction we study: target-visible verification can make invariant inference trivial even when the underlying relation is nonlinear.

Target-hidden loop verification. We study a stricter setting. The full verification instance contains a precondition, loop, and post-loop target Q , but the policy receives only the precondition, guard, body, and in-scope variables. Assertions and `ensures` clauses are removed before generation, refinement, and Houdini pruning. The policy emits ACSL loop invariants, not a new postcondition. Only at the final gate does LOOPGYM restore the original target and ask Frama-C/WP to prove all invariant obligations and the post-loop condition. For a standard `while` loop, acceptance therefore requires

$$P \Rightarrow I, \quad \{I \wedge B\} S \{I\}, \quad I \wedge \neg B \Rightarrow Q.$$

The hidden target determines success, while the generator cannot read or paraphrase it. The task is consequently not “generate any loop invariant,” but “recover loop conditions that carry enough information to the exit to prove a held-out requirement.”

One framework for training and inference. The central design of LOOPGYM is a shared rollout–Houdini loop, rather than separate training and deployment procedures. Given the masked program, both paths sample a group of invariant rollouts, normalize and merge their clauses, optionally grow the pool through stateless WP-guided repairs, and invoke the same Houdini-style Frama-C/WP operator (Flanagan & Leino, 2001; Kirchner et al., 2015) to retain an inductive subset. Only the consumer of those survivors changes. During training, Houdini survivors are scored for behavioral coverage: generation rollouts receive standalone credit plus extra reward for hard trace units rejected by few group peers, and repairs receive credit for improving the merged pool. Malformed or non-inductive clauses are removed without erasing the value of usable clauses in the same response. A unique clause with zero sampled coverage is not penalized because it may support another survivor or the hidden target; only conservative semantic duplicates incur the redundancy cost. During inference, the final Houdini survivors are inserted into the original program and checked against the restored Q . Thus reward is coupled to exactly the rollout, pool-growth, feedback, and pruning semantics used at deployment; it is not an auxiliary objective attached to a different inference pipeline.

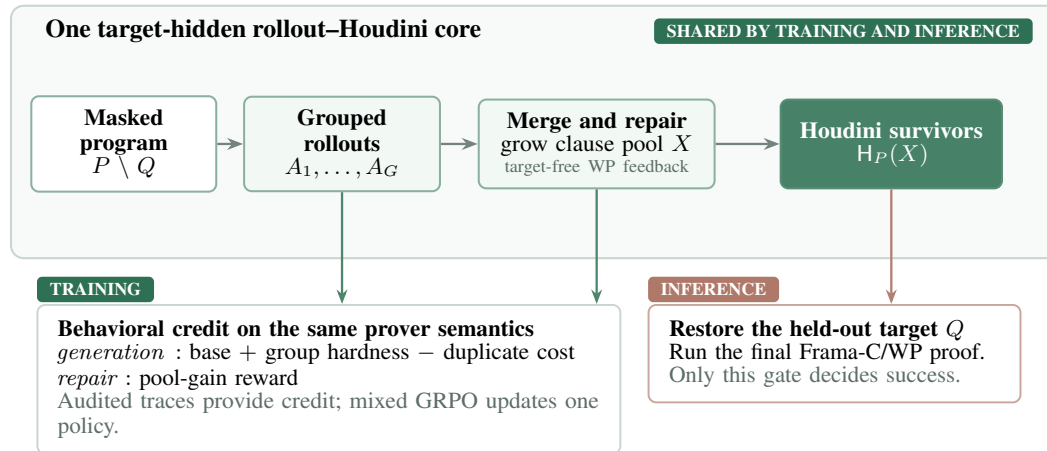


Figure 1: **Training and inference execute the same target-hidden rollout–Houdini core.** Both form rollout groups, grow a merged pool with target-free feedback, and retain clauses through the same Frama-C/WP Houdini operator. Training turns these operations into generation and refinement rewards; inference instead restores Q and applies the final proof gate. The execution sampler supplies training credit but is never called at inference.

Hiding Q removes the binary target-verification reward, while inductiveness alone remains too weak because tautologies and loose bounds pass. The execution sampler therefore constructs relational perturbations, continuations beyond a genuine exit, and sampled-range escapes without consulting Q . Candidates contradicted by observed reachability, feasible fresh entries, or incomplete state coverage are suppressed. These audited traces provide the dense strength signal attached to the shared framework, while the held-out target remains the authoritative final criterion.

Contributions. This paper makes the following contributions:

- **A target-hidden verification task.** We separate the policy view from the verification instance: the model generates invariants without Q , while acceptance still requires proving Q after the loop (§3).
- **A shared rollout–Houdini framework.** Training and inference use the same grouped rollouts, growing clause pools, stateless WP feedback, and prover-backed Houdini survivors; training attaches rewards, while inference attaches the restored final proof obligation.
- **An execution-grounded objective.** Generation receives standalone negative coverage and positive hardness credit for trace units rejected by few group peers; refinement receives its gain over the shared pool. Response caps and conservative duplicate/overflow costs prevent clause accumulation, while three conservative negative families provide target-free strength evidence (§4).
- **An auditable evaluation contract.** We define controlled target-visibility, equal-budget inference, and reward ablations, and validate the implemented signal with known-quality discrimination and a full-suite label audit (§5).

2 RELATED WORK

Loop-invariant generation. Classical techniques infer invariants with abstract domains, templates, interpolation, or recurrences (Cousot & Cousot, 1977; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017); data-driven systems instead infer likely relations from traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). Recent LLM systems broaden the candidate language. LOOPY generates ACSL clauses, filters them with Houdini, and repairs failures using Frama-C feedback (Kamath et al., 2023); CLAUSE2INV generates clauses and combines them under counterexample guidance (Cao et al., 2025). IRANK addresses the cost of sampling by contrastively re-ranking LLM-generated invariants before verifier calls (Chakraborty et al., 2023). These systems improve candidate generation, combination, or ordering for a supplied verification problem. By contrast, LOOPGYM withholds the post-loop target Q throughout generation and training, then accepts an invariant set only if it is inductive *and* proves the restored obligation $I \wedge \neg B \Rightarrow Q$. Houdini (Flanagan & Leino, 2001) is therefore a sound pruning component in our pipeline, not the generation objective: it can remove non-inductive clauses from a fixed pool, but cannot determine whether the surviving pool retains enough information for an unseen Q . Our additional distinction is alignment: grouped rollout construction and Houdini survival define one shared core for both policy training and deployment inference.

Specification generation. LLM-based tools also synthesize artifacts broader than loop invariants. AUTOSPEC decomposes C programs, repeatedly samples ACSL contracts and annotations, and retains verifier-accepted clauses (Wen et al., 2024); SPECGEN combines LLM proposals with mutation in Java/JML (Ma et al., 2025). SESPEC uses symbolic execution to expose path-sensitive strongest-postcondition information and generate goal-independent, verifier-checked specifications for C (Yang et al., 2026). These systems aim to recover function-level behavioral specifications, possibly including preconditions, postconditions, predicates, and loop annotations. LOOPGYM studies a narrower but deliberately controlled problem: the output is an ACSL loop-invariant set, the downstream target is hidden rather than generated, and success is measured by whether the set is sufficient for that held-out target. Thus our focus is not general contract recovery, but learning target-agnostic loop facts that remain useful when a concrete verification obligation is later restored.

Reinforcement learning from formal feedback. Earlier invariant learners use solver feedback to train policies or differentiable predicates (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al.,

2020). RE:FORM reduces human priors in Dafny generation through scalable data curation and verifier-guided RL (Yan et al., 2025). SPECRL goes beyond binary verifier acceptance: it generates impossible input–output pairs and rewards specifications that reject them, providing a completeness signal for Dafny specification synthesis (Huang et al., 2026). LOOPGYM shares the idea that formal validity alone cannot distinguish a useful specification from a weak one, but specializes it to hidden-target loops. Its negatives are relational perturbations, over-run exit continuations, and sampled-range escapes over internal loop states; conservative feasibility and coverage guards suppress uncertain labels, a broader execution audit checks for collisions, and Frama-C/WP separately proves inductiveness. Generation reward combines standalone negative rejection with positive hardness credit for trace units rejected by few group peers, while charging only conservative semantic duplicates; repair reward measures the gain of a capped response over the same merged pool consumed by inference. The held-out Q is never used as a feature or reward label and appears only in final acceptance.

3 PROBLEM SETTING

3.1 TARGET-HIDDEN EXIT SUFFICIENCY

Fix a verification instance (P, Q) , where $P = (\phi_{\text{pre}}, B, S)$ contains the precondition, loop guard, and body, and Q is a post-loop assertion or function postcondition. Let $\mathcal{R}(P)$ be the states reachable at the loop head from an initial state satisfying ϕ_{pre} . A predicate I is *inductive* when

$$\phi_{\text{pre}} \Rightarrow I \quad \text{and} \quad \{I \wedge B\} S \{I\}.$$

Inductiveness gives a sound over-approximation of $\mathcal{R}(P)$, but does not make it useful for a particular verification task. We call I *exit-sufficient* for Q when

$$I \wedge \neg B \Rightarrow Q.$$

Definition 1 (Target-hidden loop verification). *The verification framework receives (P, Q) , but the policy receives only P . It must generate an invariant set I without observing Q , and succeeds only when I is both inductive and exit-sufficient for the original Q .*

The restriction is on information flow, not output syntax: both visible and hidden systems emit ACSL `loop invariant` clauses. In the visible setting, however, Q may itself be inductive and can simply be copied into the loop, as in `NLA_lipus/10.c`. Target hiding removes this shortcut. It also makes binary verification unusable as a training reward, since querying the held-out target during RL would leak the answer. LOOPGYM instead rewards target-free behavioral strength and retains the original Q as the sole final acceptance criterion.

3.2 A SHARED CANDIDATE-PROCESSING OPERATOR

For a rollout group $\mathcal{A} = \{A_1, \dots, A_G\}$ and response-cap hyperparameter K , each policy response is first capped at K parsed invariant clauses; let A_i denote that ordered prefix and $X_0 = \bigcup_i A_i$ its normalized, deduplicated clause union. Each optional repair round receives only target-free WP verdicts and grows the pool monotonically,

$$X_{t+1} = X_t \cup R_t.$$

Finally, $H_P(X_t)$ denotes the subset that survives syntax checking and the Frama-C/WP Houdini fixpoint on the masked program P . The sequence

$$\mathcal{A} \longrightarrow X_0 \longrightarrow X_1, \dots, X_m \longrightarrow H_P(X_m)$$

is the shared rollout–Houdini core in Figure 1.

Training and inference differ only in how they consume this operator. Training evaluates individual rollouts, their union, and repaired pools to assign generation and refinement credit. Standalone rejection measures strength, while group-relative hardness rewards trace units rejected by few other rollouts. The only redundancy cost applies to conservative semantic duplicates; unique clauses are never punished merely for overlapping sampled coverage. Inference performs the same union, pool growth, and Houdini pruning without constructing reward traces, then restores Q for final verification. Training uses sampled positives as a cheap pre-filter, but soundness in both modes comes from the same Frama-C/WP Houdini implementation. Thus deployment is independent of the reward service while preserving the candidate and prover semantics optimized during training.

4 METHOD

Notation. For a verification instance (P, Q) with $P = (\phi_{\text{pre}}, B, S)$, a *rollout* $A = (a_1, \dots, a_L)$ is the ordered sequence of parsed invariant clauses produced by one policy sample, and a *group* $\mathcal{A} = \{A_1, \dots, A_G\}$ is the set of rollouts sampled for P in one RL step. The response cap is the hyperparameter K ; $\bar{A} = (a_1, \dots, a_{\min(L, K)})$ is the accepted prefix and $o(A) = \max(0, L - K)$ its overflow. The sampler emits positives $\mathcal{P}$ (sampled reachable loop-head states, each carrying its run’s input valuation) and synthetic negative candidates $\mathcal{N}$ grouped into *trace units*: each unit is one perturbed history under a concrete input, witnessed by one or more states. $H_P(X)$ denotes the subset of clause set X that survives the shared rollout–Houdini core on masked program P , and $\text{rej}(X) \subseteq \mathcal{N}$ the trace units rejected by $H_P(X)$ — a unit is rejected iff some surviving clause is false at one of its witness states.

4.1 TARGET MASKING AND ACCEPTANCE

The implementation maintains two program views. The *original view* contains the input `requires` clauses and every verification target, including post-loop assertions, in-loop assertions, and function `ensures`. The *masked view* preserves the C program and `requires` clauses but removes complete `assert/ensures` clauses, including quantified targets. Generation, refinement feedback, and Houdini pruning operate only on the masked view. The sampler independently parses the loop and never consults the stored target.

The policy output is a list of scalar ACSL clauses of the form `loop invariant <expr>;`. Only the first K parsed clauses of each generation or refinement response are admitted; later clauses never enter a prover pool. These clauses are inserted before the loop together with the inferred `loop assigns` set. Candidate acceptance has two stages. First, Frama-C/WP checks one establishment and one preservation obligation per invariant; Houdini iteratively removes failed clauses. Second, the surviving set is inserted into the original view. The final result is successful only if the source parses, every expected invariant obligation succeeds, and every non-invariant WP goal induced by the restored assertions and postconditions succeeds. Thus the target is hidden from all model-facing stages but remains authoritative for evaluation.

4.2 THE SHARED ROLLOUT–HOUDINI CORE

Training and inference share one candidate-processing operator. Given a rollout group $\mathcal{A} = \{A_1, \dots, A_G\}$, both cap each response, normalize and deduplicate its accepted clauses, form the union $U = \bigcup_i \bar{A}_i$, and use the same Frama-C adapter to compute inductive survivors. The authoritative operator $H_P(X)$ first uses a Frama-C syntax scrub to remove the exact malformed invariant lines, then runs Houdini over Frama-C/WP (Flanagan & Leino, 2001; Kirchner et al., 2015) to iteratively remove clauses whose establishment or preservation goals fail. During reward computation, a positive pre-filter first removes out-of-scope clauses and clauses falsified by sampled reachable states. Inference supplies no sampled states and skips this optimization. Because both modes end in the same prover-backed Houdini operator, sampled states improve training efficiency but do not establish soundness.

The two modes differ only in how they consume this common operator. Generation training evaluates $H_P(\bar{A}_i)$ for every rollout and $H_P(U)$ for the group. Refinement training starts from a merged pool X and compares $H_P(X \cup R_i)$ with $H_P(X)$ for sampled repairs R_i . Inference forms the same union, optionally repeats the same stateless WP-verdict prompt and monotone update $X \leftarrow X \cup R_i$, and returns $H_P(X)$ to the final target-bearing verifier. Thus the reward service and deployment executable remain operationally independent, but rollout grouping, pool semantics, feedback format, and Houdini survival are shared by construction.

4.3 SAMPLER: POSITIVES AND AUDITED NEGATIVE CANDIDATES

The sampler is deliberately blind to any postcondition or assertion: it executes only the precondition-respecting loop behavior needed for trace collection. The current parser accepts one braced `while` loop over scalar C `int` parameters, locals, and file-scope variables; unsupported multi-loop/`for`/pointer inputs fail explicitly, while incomplete body state or calls trigger a positive-only

fallback. It instruments the loop to print the loop-head valuation on entry, for a dense 512-iteration prefix followed by periodic eight-state bursts, and at exit; compiles with `gcc`; and executes over a deterministic, precondition-checked input sweep (edge and moderate magnitudes, systematic tier combinations, plus a few large-magnitude probes that widen the input envelope). Loops run to their real exit; runs that hit the iteration cap are flagged and excluded from any range conclusion. The observed loop-head states are the positives.

The sampler constructs negative *candidates*, not a proof of global unreachability. Family-specific evidence and conservative guards reduce label risk, while the full-suite collision audit in §5 checks the emitted labels against broader executions.

- **Relational perturbations** apply $\pm\{1, 2\}$ single-axis and pairwise steps to trace states whose next three coordinates were printed. Candidates colliding with any sampled reachable valuation are removed; the survivors expose missing laws such as $x + y = n$.
- **Over-runs** execute the real body past an observed genuine exit, preserving transition relations while moving beyond that run’s exit; each continuation forms one trace unit.
- **Escapes** move one variable beyond its sampled range by ladder steps. Together with over-runs they probe missing range facts without claiming that a finite sample covers all executions.

Construction guards reduce label risk — a mislabeled (actually reachable) negative would distort the tightness signal. States observed in any run are never negatives; states that could constitute a fresh loop entry for admissible inputs (parameters free subject to `requires`, initialized locals at their initializers) are dropped; and a loop whose guard or body is controlled by `unknown()` emits positives but no synthetic negatives because finite oracle runs under-approximate its reachable set. The same no-negative fallback applies to untracked block-local state, pointer/array state, and function calls in the body. For deterministic loops, a capped run additionally disables escapes because its sampled range is incomplete. The sampler thus degrades conservatively: where state/control coverage is incomplete it emits no synthetic negatives rather than guessing labels.

4.4 TRAINING THE SHARED CORE

Let $R_i = \text{rej}(\bar{A}_i)$. Standalone coverage and the rarity of each rejected trace unit are

$$\text{base}(A_i) = \frac{|R_i|}{|\mathcal{N}|}, \quad (1)$$

$$f(n) = \sum_{j=1}^G \mathbf{1}[n \in R_j], \quad h(n) = 1 - \frac{f(n)}{G}, \quad (2)$$

$$\text{hard}(A_i) = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} h(n). \quad (3)$$

All rejection sets are computed after standalone Houdini, so malformed and non-inductive clauses are discarded without zeroing the useful remainder of a response. Base rewards overall strength; hard assigns additional positive credit to trace units rejected by few other rollouts in the group. This uses group diversity without requiring any rollout to contain a complete proof.

We do not penalize a unique clause merely because it adds no independently observed negative coverage. Such a clause may be required to keep another clause inductive or may encode a direction needed by the hidden target Q . The only within-cap redundancy cost is

$$d_{\text{dup}}(A) = |\bar{A}| - |\text{dedup}(\bar{A})|.$$

Here `dedup` uses a conservative, solver-free AST key for the scalar-integer language. It normalizes parentheses, comparison direction, equality sides, immediate commutative operands, harmless identities, and order-preserving Boolean association. It deliberately does not reorder Boolean terms, reassociate arithmetic, cancel factors, move terms, or rewrite division and remainder; unsupported syntax falls back to normalized text. The first spelling is retained. A removed occurrence therefore cannot add a Houdini survivor or final proof fact under the supported semantics.

The complete generation reward is

$$r_{\text{gen}}(A) = w_b \text{base}(A) + w_h \text{hard}(A) - w_r d_{\text{dup}}(A) - w_o o(A), \quad (4)$$

where $w_b, w_h, w_r, w_o \geq 0$ are reward weights; the defaults are (1.0, 0.3, 0.02, 0.05). The cap makes clauses after position K ineligible for coverage credit; when $w_o > 0$, the linear overflow term makes continued invariant emission strictly costly.

Rejection tests are pure Python; division and remainder use C-style truncation on concrete integer states, while the only heavyweight cost is the prover. The per-rollout and union clause sets are memoized and verified in parallel, and each WP run discharges its goals concurrently. The group additionally receives $\text{batch} = |\text{rej}(U)|/|N|$ and a re-roll flag when it falls below a threshold. If no negative trace unit can be constructed safely, rollout and batch signals fall back to binary Frma-C/WP validation: a nonempty candidate set receives one iff every clause survives standalone Houdini, and zero otherwise.

Refinement training is a separate group type, not a third generation-reward term. Given a shared merged pool X and repair response R_i , its reward is

$$\Delta \text{base}_i = \text{base}(X \cup R_i) - \text{base}(X), \quad (5)$$

$$r_{\text{ref},i} = \Delta \text{base}_i - w_o o(R_i). \quad (6)$$

Here R_i denotes the capped response prefix when joined to X ; the existing pool itself is not truncated. The pool only grows, so the diagnostic delta is nonnegative. Within the cap, copied, trivial, or broken repairs that add no discrimination after H_P receive zero; an overlong response can receive negative training reward through its overflow cost. Generation and refinement groups are mixed in one policy update, directly training the two model actions exercised by deployment.

4.5 INFERENCE THROUGH THE SHARED CORE

At inference time the rollout provider receives the masked program and returns n candidate invariant sequences, each capped at K clauses before union. The framework applies the same normalization and union operation used in training; unlike reward computation, inference never invokes the execution sampler or constructs positive and negative examples. When $m > 0$, each refinement round runs a syntax check and at most two Frma-C/WP passes on the masked program. The second pass is needed only after removing establishment failures that could mask preservation failures. The resulting verdict table reports syntax and scope errors and distinguishes failed establishment from failed preservation, but contains no target-derived feedback. The stateless prompt is the same template used to form refinement-training groups and asks the provider to repair rejected clauses or add supporting companions; each refinement response is likewise capped at K clauses before joining the pool. New candidates join the pool and originals remain, so a later companion may rescue an earlier clause. Refinement stops when no clause is rejected, the pool reaches a fixpoint, or the m -round budget is exhausted; $m = 0$ recovers the plain pipeline.

The shared H_P operator then performs full Houdini pruning and produces the final invariant set on the masked view. Only then does the framework annotate the original target-bearing source and run Frma-C/WP. The verifier separately checks that the number of invariant results matches the number of inserted clauses and that all assertion, postcondition, and contract goals succeed. If final verification fails, inference may request another rollout group up to its re-roll budget; candidate ranking prefers a verified result and then the result with more surviving invariants. Because refinement only enlarges the pool and the same final proof gate remains authoritative, enabling refinement cannot remove a clause accepted by the plain pipeline.

5 EXPERIMENTS

We organize the evaluation around three research questions. RQ1 probes why target-hidden RL must preserve useful exploration rather than optimize soundness alone. RQ2 evaluates the effectiveness and cost of the inference framework against existing tools, with no LoopGym-trained policy involved. RQ3 then tests whether framework-aligned RL improves the grouped rollout-combine-Houdini objective even when standalone $\text{pass}@k$ does not. This separation follows prior invariant-synthesis evaluations that distinguish candidate generation, candidate combination, and end-to-end tool performance (Kamath et al., 2023; Cao et al., 2025; Wei et al., 2025).

5.1 EXPERIMENTAL SETUP

Target-hidden acceptance. Before any model-facing or intermediate generation step, we remove ACSL `assert` and `ensures` predicates, executable assertion calls, and conventional error-label names. Preconditions and executable loop semantics remain visible. After a method fixes its loop invariants, we insert them into the untouched original program and restore the target. A task succeeds iff the source parses, Frama-C/WP proves establishment and preservation for every retained invariant, and all restored assertion or postcondition goals succeed. Failures, timeouts, malformed outputs, and unsupported inputs count as zero.

Verifier and model configurations. All final checks use Frama-C 32.1 (Germanium), WP, and Z3 4.15.4. The RQ1 probe samples 100 target-hidden responses per task from the official Qwen3-4B, Qwen3-8B, Qwen3-14B, and Qwen3-30B-A3B checkpoints (Yang et al., 2025), at temperature 1.0 with thinking disabled. RQ2 fixes `gpt-5-nano` across its LLM-backed rows; differences among the LoopGym rows are therefore inference-only. RQ3 compares Qwen3-8B with the corresponding 8B-RL-Zero checkpoint under the same 100-rollout protocol.

5.2 RQ1: WHY MUST THE REWARD PRESERVE EXPLORATION?

Soundness is a degenerate reward. In target-hidden training, the exact target Q is unavailable, so the most obvious verifier reward is whether a candidate satisfies initiation and preservation. This signal cannot distinguish a strong loop summary from `true`, a loose bound, or another weak-but-sound invariant. Under a binary whole-response reward, these easy modes receive the same return as a semantically strong invariant; once they dominate a rollout group, normalized group advantages vanish and training has no incentive to discover stronger relations. A single malformed or non-inductive clause can also zero an otherwise useful response, encouraging short and conservative outputs.

This failure is compounded by distribution sharpening. The NeurIPS 2025 Best Paper Runner-Up study of Yue et al. shows that RLVR can improve low- k accuracy by concentrating probability mass on already rewarded reasoning paths while narrowing the reasoning coverage exposed at large k (Yue et al., 2025). Such sharpening is particularly harmful here: grouped inference depends on different rollouts contributing complementary invariant clauses, not merely repeating the most likely sound clause.

Measuring complete and clause-level exploration. For each program, let c be the number of independently verifying responses among $n = 100$. Direct `pass@ $k$` uses the unbiased estimator $1 - \binom{n-c}{k} / \binom{n}{k}$ and asks whether some rollout independently contains a complete proof. `Combine@ $k$` instead deduplicates all clauses in the first k rollouts, applies iterative Houdini pruning, and checks whether the surviving pool proves the restored target. It therefore asks whether the model has explored enough complementary proof ingredients for the inference framework to assemble a proof.

Figure 2 exposes two limitations. First, small models rarely assemble a complete invariant set: direct `pass@100` is 15.62% for 4B, compared with 33.29% for 8B and 39.90% for 14B. Second, these weak complete responses still contain useful pieces. Union and Houdini raise 4B to 47.84% and 14B to 58.05% at $k = 100$. Combination thus recovers substantial value from partial rollouts, but it does not erase the exploration gap: it cannot invent a relation that the policy never emits.

Exploration also saturates early. From `combine@30` to `combine@100`, no model gains more than 0.60 points, indicating that later responses contribute little new proof-relevant coverage. We summarize rollout efficiency by

$$k_{95} = \min\{k : C_{\pi}(k) \geq 0.95 C_{\pi}(100)\},$$

where $C_{\pi}(k)$ is the verification rate after union and Houdini over k policy samples. A successful training objective should move this curve both upward and leftward: raise the final exploration ceiling while reaching it with fewer model generations.

Reward implication. LoopGym consequently treats inductiveness as a feasibility gate, not as the complete reward. Malformed lines are discarded and non-inductive clauses are removed by the same Houdini operator used at inference; the remaining clauses receive standalone strength credit, with extra positive hardness credit for trace units rejected by few other rollouts. Weak clauses add no

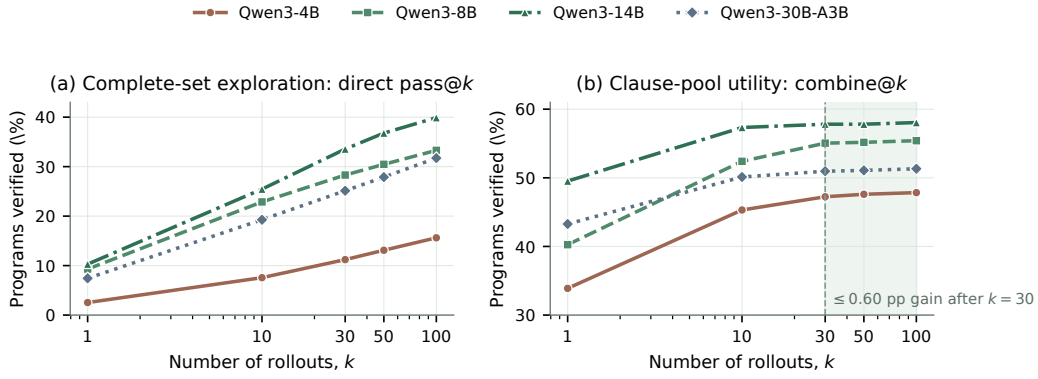


Figure 2: Exploration probe over four official Qwen3 checkpoints. Direct pass@ k measures complete responses, whereas combine@ k measures whether the accumulated clause pool contains a sufficient Houdini-surviving subset. The combine curves change by at most 0.60 percentage points after $k = 30$.

coverage gain, while conservative semantic duplicates receive a small explicit cost. This preserves the useful part of an imperfect response while rewarding exploration without requiring a complete response-level proof. Combination is therefore an exploration multiplier, not an exploration substitute.

The reward sanity checks support this distinction without claiming a trained policy result. Across ten hand-written specifications, every gold clause survives and the minimum gold base score is 0.996, with zero discrimination violations. A larger audit over Core-366 covers 486,302 candidate negative states and finds no hard pair-level collision under the expanded sampling check.

5.3 RQ2: HOW EFFECTIVE AND EFFICIENT IS THE INFERENCE FRAMEWORK?

Benchmark. We evaluate the inference framework on 832 integer C programs. The *Core-366* subset contains 316 linear programs drawn from Code2Inv, SyGuS, and SV-COMP and 50 nonlinear programs from LIPuS. The remaining 466 programs are the integer fragment of Loopy’s original 469-program corpus; three floating-point programs are excluded because they fall outside our scalar-integer predicate language. Every method uses the same fixed manifest, and unsupported inputs remain failures in the common denominator.

RQ2 evaluates inference, not model training. All LoopGym variants use the same gpt-5-nano checkpoint. NAIVE uses one response with a minimal prompt; R1-NOH changes only to the LoopGym prompt; R1-H additionally applies Houdini; and R5-H/R10-H union five/ten target-hidden rollouts before the same Houdini and final-WP gate. Re-rolling and refinement are disabled.

We compare with the native target-hidden pipelines of AUTOSPEC (Wen et al., 2024), SESPEC (Yang et al., 2026), and CLAUSE2INV (Cao et al., 2025), plus Daikon 5.8.24 (Ernst et al., 2007). Clause2Inv supports only the 366 programs with transition VCs; its 466 unsupported inputs remain failures in the common denominator.

Table 1 shows that R5-H verifies 622 programs, 113 more than AutoSpec, the strongest external baseline. The controlled rows attribute the gain to the prompt, Houdini pruning, and multi-rollout union: 379/832 for NAIVE, 452/832 for R1-NOH, 504/832 for R1-H, and 622/832 for R5-H. The improvement costs generation: R5-H averages 33,404.56 exact tokens and 284.81 seconds per task; R10-H adds only 16 proofs while rising to 68,953.35 tokens and 536.81 seconds. This diminishing return motivates improving exploration efficiency rather than scaling the rollout budget indefinitely.

5.4 RQ3: DOES FRAMEWORK-ALIGNED RL IMPROVE GROUPED INFERENCE?

In its normal negative-coverage mode, the LoopGym reward is deliberately not a binary verdict on an entire response. Such a verdict discards all useful information when one clause is malformed or non-inductive and encourages the weak conservative modes identified in RQ1. Instead, syntax

Method	Linear / 316	NLA / 50	Loopy / 466	All / 832	Time (s)	Exact tokens	Est. tokens
AUTOSPEC	203	4	302	509 (61.18%)	64.63	—	2,031.54
CLAUSE2INV	63	1	0	64 (7.69%)	29.04	—	385.80
DAIKON	73	0	101	174 (20.91%)	4.89	0	—
SESPEC	180	3	231	414 (49.76%)	117.31	44,807.05	—
NAIVE	154	4	221	379 (45.55%)	28.87	4,493.37	—
R1-NoH	193	7	252	452 (54.33%)	53.52	5,849.64	630.14
R1-H	203	7	294	504 (60.58%)	66.60	6,928.30	—
LOOPGYM (R5-H)	250	7	365	622 (74.76%)	284.81	33,404.56	—
R10-H	255	7	376	638 (76.68%)	536.81	68,953.35	—

Table 1: Target-hidden inference effectiveness and mean per-task generation cost. Exact and estimated token averages are reported separately. R1-NoH’s time and exact-token means cover Loopy-466, while its estimate covers Core-366; unsupported Clause2Inv inputs are not timed but remain failures in the common 832-program denominator.

scrubbing and Houdini remove failed clauses, after which the reward measures the standalone strength of the survivors and gives extra positive credit to hard trace units rejected by few group peers. RQ3 tests the consequence of this alignment by comparing Qwen3-8B with 8B-RL-Zero under identical inference settings.

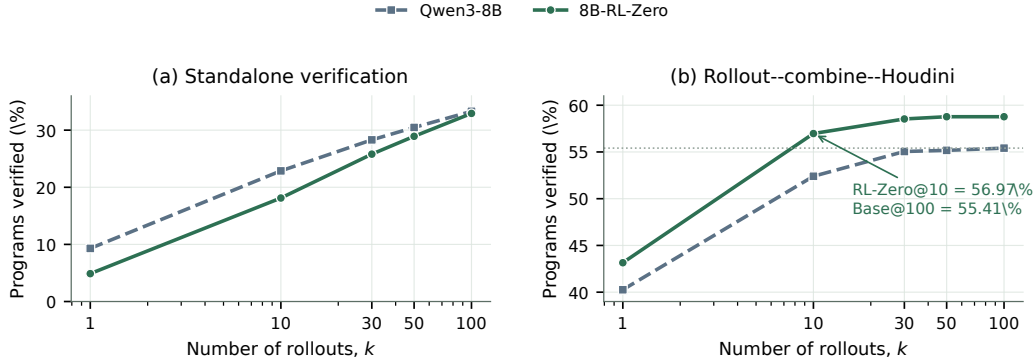


Figure 3: Framework-aligned RL improves grouped inference rather than standalone success. RL-Zero slightly decreases direct pass@ k (left) but consistently improves combine@ k (right). With ten rollouts, RL-Zero exceeds the base model’s result with 100 rollouts.

Figure 3 shows a sharp divergence between the two metrics. RL-Zero lowers direct pass@1 from 9.30% to 4.88% and leaves direct pass@100 nearly unchanged (33.29% versus 32.93%). Judged only by standalone verification, the trained policy would appear worse. Under the deployed pipeline, however, combine@10 rises from 52.40% to 56.97%, and combine@100 rises from 55.41% to 58.77%. Ten RL-Zero rollouts already outperform 100 base-model rollouts, while measured k_{95} falls from 30 to 10.

The RL policy therefore moves the combine curve both upward and leftward: it raises the attainable verification rate and reaches that rate with fewer model generations. This is the intended training outcome. The objective does not require every rollout to verify independently; it rewards the usable clauses that survive pruning and cover difficult trace units, matching the diverse clause supply consumed by rollout–combine–Houdini at deployment.

6 CONCLUSION

Loop verification requires invariants that are both inductive and strong enough to establish the condition required after the loop. When that condition is visible, generation may collapse to copying

the assertion, as illustrated by `NLA_lipus/10.c`. We instead study target-hidden verification: the policy sees the loop but not its assertion, while the original assertion remains the final acceptance criterion.

LOOPGYM centers training and inference on the same rollout–Houdini framework. Both paths sample groups of candidate invariant sets, merge and optionally repair a growing clause pool, and retain its inductive core with the same Frama-C/WP Houdini operator. The training objective assigns standalone rejection and positive hardness credit for trace units rejected by few rollouts in the group. A unique clause with zero sampled coverage is not penalized because it may support another invariant or the held-out target; only conservative semantic duplicates receive a redundancy cost. A configurable K -clause response cap with overflow cost prevents unbounded output growth. Refinement receives pool-improvement credit subject to the same response cap; inference replaces these rewards with final target-bearing verification. This shared scaffold aligns learning with deployment while keeping Q absent from prompts, feedback, pruning, and reward labels. The pipeline therefore rewards semantic information carried through the loop without weakening the end goal: a result succeeds only when the restored postcondition verifies.

Beyond the immediate system, we see the shared training–inference scaffold and audited behavioral rewards as portable. Audited synthetic negatives generalize to domains where construction-specific evidence is cheaper than proving optimality; keeping assumptions and collision checks explicit is preferable to compensating for bad labels in the scoring formula. Future work includes multi-loop and heap-manipulating programs (Liu et al., 2024a), richer assertion languages, and conditions that connect several loops before a final postcondition.

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 84–96, 1978.
- Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 443–456, 2013.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

-
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.
- Carlo Alberto Furia and Bertrand Meyer. Inferring loop invariants using postconditions. In *Fields of Logic and Computation: Essays Dedicated to Yuri Gurevich*, pp. 277–300. Springer, 2010.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.
- Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.
- Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 281–292, 2008.
- Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 634–640. Springer, 2009.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.
- Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- Jeffrey C. Lagarias. The $3x + 1$ problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985. doi: 10.1080/00029890.1985.11971528.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiong Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.
- Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.
- ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.

-
- ThanhVu Nguyen, Timos Antonopoulos, Andrew Ruef, and Michael Hicks. Counterexample-guided approach to finding numerical invariants. In *Proc. 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pp. 605–615, 2017.
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 318–329, 2004.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.
- Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024.
- Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026.
- Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A CANONICAL COMPONENT DEFAULTS

Table 2 lists the implemented defaults. Execution-trace sampling belongs only to reward construction, while grouped model rollouts, monotone refinement pools, and the Houdini backend are shared with inference. Inference has its own refinement budget and never constructs a trace example set.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 values in $[-64, 64]$	deterministic edge-first magnitude schedule
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	$\pm\{1, 2\}$	single-axis and pairwise perturbations off dense bases
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
escape ladder	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
over-run steps	24	post-exit continuation length (one trace unit per run)
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_b, w_h)	(1.0, 0.3)	standalone coverage / group-hardness weights
w_r	0.02	cost per conservative semantic duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generation/refinement response
w_o	0.05	overflow cost per clause after K
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget
m_{refine}	0	inference refinement disabled by default; any non-negative round budget is supported

Table 2: Implemented defaults of the sampler (§4.3), reward (§4.4), inference, and verification backend.

B GENERATION PROMPT

The complete generation template; closed-book inference strips an assertion when one is present before inserting the program source. General ACSL rules are supplied separately by `system_prompt.txt`.

Generation prompt

You are given a C function. Produce the strongest inductive ACSL loop invariants that capture the loop’s behavior (conservation/relational laws + tight progress bounds). Output ONLY invariant lines, each formatted exactly as:
 loop invariant <expr>;
 Output at most 20 invariant lines.

Program:
 {program}

C REWARD SERVICE INTERFACE

The reward framework is packaged as a self-contained, CPU-only container with gcc, Frama-C/WP, Why3, and Z3 bundled. It exposes `POST /reward`, accepting program source and a rollout array, with the following response fields:

```
{program, n_positives, n_negatives, filter_mode,
 reward_mode, rollout_rewards[], base[], hard_bonus[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score, should_reroll}
```

Rollouts may be raw LLM text, invariant lists, or annotated code; parsing is uniform. For refinement training, `POST /refine-feedback` returns the WP verdict table and assembled stateless prompt, while `POST /refine-reward` returns both the nonnegative diagnostic Δ_{base} and `refine_rewards[]` after overflow cost. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.